universidade de aveiro
Departamento de Eletrónica, Telecomunicações e Informática

Curso:	Mestrado em Engenharia de Computadores e Telemática
Disciplina:	42078 - Informação e Codificação
Ano letivo:	2025/2026

Relatório

Lab work n2

Autores: 103075 — José Jordão
108689 — Gabriel Janicas
107715 — Afonso Rodrigues

Turma: P1

Docente: Armando J. Pinho

Universidade de Aveiro, November 17, 2025

Contents

List of Figures

List of Tables

Chapter 1

Important Information

All parts of this laboratory work were developed collaboratively, with tasks and responsibilities being evenly distributed among all members of the group.

All C++ programs developed for this lab are managed by a single `Makefile`. To compile all executables, navigate to the root directory of the project in a terminal and run the following command:

```
make
```

To clean all compiled object files, executables, and the contents of the `Result_Images/` directory, run:

```
make clean
```

Chapter 2

Part 1

Part I covers fundamental image manipulation using OpenCV. Program 1.1 extracts B, G, or R color channels into single-channel images. Program 1.2 implements manual, pixel-level transformations (negative, mirror, rotation, intensity) without using OpenCV's high-level functions.

2.1 Exercise 1

2.1.1 Methodology

The program loads a 3-channel (BGR) image into an OpenCV cv::Mat structure. A new, 8-bit single-channel (grayscale) image of the same dimensions is then created. The core logic iterates through every pixel coordinate (r, c) of the source image. At each coordinate, it reads the 3-channel Vec3b (BGR) value and writes only the selected channel's intensity value (e.g., bgr_pixel[2] for Red) to the corresponding (r, c) coordinate in the new single-channel destination image.

2.1.2 Usage

The program is executed from the command line, taking an input filename, output filename, and a channel ID as arguments. The program automatically reads from the imagens_PPM/ directory and writes to the Result_Images/ directory.

```
1
2    ./1.1_extract_channel [input_filename.ppm] [output_filename.png] [
    channel_id]
3
```

2.1.3 Results

This process successfully isolates the intensity map of a single color channel, as demonstrated with the tulips.ppm image. The results for each channel extraction confirm the program's logic.

Red Channel (Channel 2): The resulting image correctly shows the bright red and yellow tulips as the brightest areas (almost white). This is the expected behavior, as both pure red and pure yellow (a mix of red and green) have high-intensity red-component values. The green stems and leaves appear very dark.

Figure 2.1: Original Tulips Image

Figure 2.2: Red channel extraction

Blue Channel (Channel 0): The entire image appears very dark. This is also correct, as the subject (red flowers, yellow flowers, and green leaves) contains almost no

blue-component values.

Figure 2.3: Blue channel extraction

Green Channel (Channel 1): In this extraction, the green leaves and stems are very bright. The yellow tulips are also bright, as they are a mix of red and green. The pure red tulips, however, become very dark, as they lack a green component.

Figure 2.4: Green channel extraction

2.2 Exercise 2

2.2.1 Methodology

This program implements several transformations by directly manipulating pixel data. As per the lab requirement, high-level OpenCV functions that perform these tasks automatically (e.g., cv::flip, cv::rotate, cv::subtract) were not used.

Instead, the methodology relies only on basic OpenCV functions for file I/O (cv::imread, cv::imwrite) and direct pixel access (cv::Mat::at). The transformation logic itself is custom-written and applied pixel-by-pixel:

(a) Negative: The negative is calculated on a per-channel basis. The formula new_value = 255 - old_value is applied to each of the B, G, and R components of every pixel.

(b) Mirroring: This operation re-maps pixel coordinates from the source to the destination. For horizontal mirroring, the pixel at (r, c) is written to the new location (r, width - 1 - c). For vertical mirroring, (r, c) is mapped to (height - 1 - r, c).

(c) Rotation (90°): This is the most complex operation, requiring a new output matrix with swapped dimensions (source width becomes destination height, and vice-versa). A 90-degree clockwise rotation maps the original pixel (r, c) to the new pixel at (c, height - 1 - r).

(d) Intensity: Intensity is adjusted by adding a constant value k to all three (B, G, R) channels of every pixel. A critical design decision was to clamp the results to the valid 8-bit range of [0, 255]. This prevents data corruption from overflow (e.g., 240 + 50 becomes 255, not 34) or underflow (e.g., 30 - 50 becomes 0, not -20).

2.2.2 Usage

The program is run from the command line:

- **Negative:**

```
./1.2_image_ops [in] [out] negative
```

- **Mirror:**

```
./1.2_image_ops [in] [out] mirror [h | v]
```

- **Rotate:**

```
./1.2_image_ops [in] [out] rotate [90 | 180 | 270]
```

- **Intensity:**

```
./1.2_image_ops [in] [out] intensity [value] (e.g., 50 or -50)
```

2.2.3 Results

The program successfully applies all transformations. The clamping logic in the intensity function is particularly important, as it prevents pixel "wrap-around" and produces the visually correct brightened or darkened image.

Processing time was measured for each operation on the 256x256 house.ppm image.

Table 2.1: Average 'user' processing time for Part 1.2 operations on a 256x256 image.

Operation	**Time** (ms)
Negative	27
Mirror (Horizontal)	36
Rotate (90-deg)	36
Intensity (+50)	34

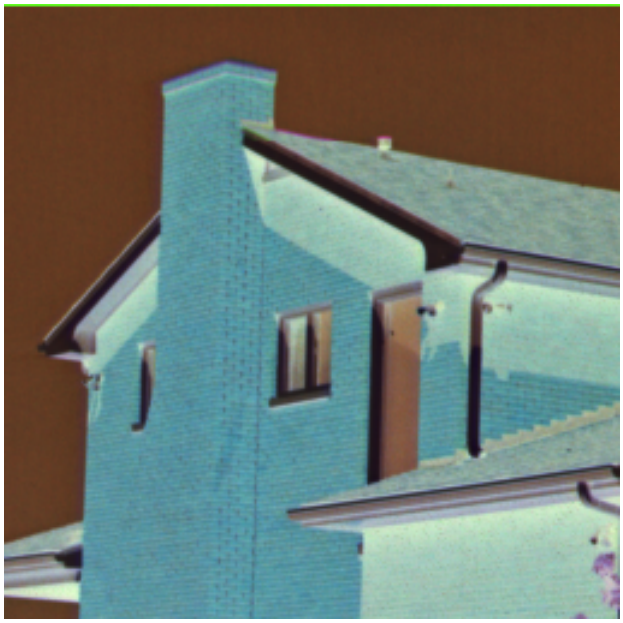

Figure 2.5: Negative mode

Figure 2.6: Horizontal mirror

Figure 2.7: 90-degree clockwise rotation

Figure 2.8: Intensity adjustment (k = +50)

Chapter 3

Part 2

Part II details the implementation of a flexible Golomb coding class. This class will be the core entropy coder for later parts, supporting a variable m parameter and two modes for handling negative numbers: sign and magnitude, and positive/negative interleaving.

3.1 Methodology

1. **BitStream Helper Class:** A separate `BitStream` class was created to handle the low-level bitwise operations. This class abstracts the logic of packing individual bits (0s and 1s) into a `std::vector<uint8_t>` for writing, and reading them back one by one for decoding. This separation of concerns keeps the `GolombCoder` class clean and focused purely on the Golomb algorithm.

2. **Golomb Algorithm:** The core `encode` function first maps the input integer to a non-negative value (using one of the modes below). It then calculates the Golomb code based on the m parameter. The quotient $q = n/m$ is encoded using a unary code (q ones followed by a zero). The remainder $r = n\%m$ is then encoded using a truncated binary representation.

3. **Negative Number Handling:** The class was designed to handle negative integers using two distinct, user-selectable modes:

    - **Sign & Magnitude:** A single sign bit (0 for positive/zero, 1 for negative) is written to the bitstream first. The Golomb code is then calculated and written for the **absolute value** of the integer.
    - **Positive/Negative Interleaving:** This method maps all integers (positive, negative, and zero) to a positive-only set. The mapping `mapped_value = (n <= 0) ? -2*n : 2*n - 1` was implemented. This non-negative `mapped_value` is then Golomb-encoded. This approach is potentially more efficient as it does not require a separate sign bit for every number.

3.2 Usage

The GolombCoder class is a software component, not a standalone executable. To validate its functionality, a dedicated test program (2.1_golomb_test, built from main_part2.cpp) was created.

The program is compiled by running the make command, which automatically links the golomb.o and bitstream.o object files with the main_part2.o file.

It is then executed by running the following command in the terminal: ./2.1_golomb_test

The test program requires no user input. It runs an automated test that encodes and decodes a sample data set and self-validates by comparing the original data to the decoded data. A "— All tests passed! —" message is printed to the console upon successful completion.

3.3 Results

The `GolombCoder` class was successfully validated using the `2.1_golomb_test` program. This program encodes a test vector of 13 integers ({`0, 1, -1, 2, -2, 3, -3, 8, -8, 15, -15, 23, -42`}) and then decodes the resulting bitstream.

The primary result is that for all tested m values (1, 5, and 8) and for both `Sign & Magnitude` and `Interleaving` modes, the decoded vector **perfectly matched** the original data. This confirms the implementation is correct, lossless, and functions as required.

A quantitative analysis was performed to measure the compression achieved by each mode on this test data, which is 416 bits uncompressed (13 integers * 32 bits/int).

Table 3.1: Golomb Compression Performance on Test Vector (416 bits uncompressed)

m **Value**	**Sign Mode**	**Compressed (bits)**	**Ratio (Uncomp/Comp)**
5	Sign & Magnitude	73	5.70
5	Interleaving	87	4.78
8 (Rice)	Sign & Magnitude	75	5.55
8 (Rice)	Interleaving	75	5.55
1 (Unary)	Sign & Magnitude	149	2.79
1 (Unary)	Interleaving	253	1.64

The results in Table 3.1 clearly show that the m parameter and sign-handling mode have a significant impact on compression.

- $m = 1$ **(Unary):** This was the least effective method, as the test data contains large-magnitude numbers (e.g., -42), which require long unary codes.
- $m = 5$ **and** $m = 8$**:** Both $m = 5$ (general Golomb) and $m = 8$ (Rice code) offered excellent compression.
- **Sign Mode Analysis:** For this specific dataset, `Sign & Magnitude` was more efficient than `Interleaving`. This is because the dataset has high-magnitude numbers (23, -42). The Interleaving method (`n' = 2*|n|...`) effectively doubles the magnitude, resulting in a larger quotient (q) and thus a longer unary code.

The `GolombCoder` class is now verified as a correct and effective entropy coder, ready for integration into the lossless codecs.

Chapter 4

Part 3

In Part III, the developed lossless audio codec employs a predictive coding scheme combined with Golomb coding of the prediction residuals. The system is designed to exploit both temporal and inter-channel redundancy in audio signals.

4.1 Methodology

Four prediction schemes were implemented and evaluated:

- **Predictor Order 0 (None):**
  - No prediction applied;
  - Residual: e[n] = x[n];
  - Serves as baseline for comparison;
- **Predictor Order 1:**
  - Linear prediction: x[n] = x[n-1];
  - Residual: e[n] = x[n] - x[n-1];
  - Low computational complexity;
- **Predictor Order 2:**
  - Linear prediction: x[n] = 2·x[n-1] - x[n-2];
  - Residual: e[n] = x[n] - (2·x[n-1] - x[n-2]);
  - Captures linear trends in the signal;
- **Predictor Order 3:**
  - Linear prediction: x[n] = 3·x[n-1] - 3·x[n-2] + x[n-3];
  - Residual: e[n] = x[n] - (3·x[n-1] - 3·x[n-2] + x[n-3]);
  - Captures quadratic trends;

Inter-channel Decorrelation (Stereo)

For stereo files, spatial redundancy between the Left and Right channels is removed using a Mid-Side (M/S) transform. This re-encodes the L/R channels into a 'Mid' (average) channel and a 'Side' (difference) channel:

- `Mid` $= (L + R) \gg 1$
- `Side` $= L - R$

The 'Mid' channel is similar to a mono signal, while the 'Side' channel is a residual signal, both of which are highly compressible.

Entropy Coding

The resulting residuals (from prediction and/or M/S) follow a Laplacian-like distribution centered at zero. These are efficiently compressed using the Golomb Coder (from Part 2). The codec supports two modes for the m parameter:

- **Fixed m:** A user-specified m value.
- **Adaptive m:** The preferred mode, where the codec calculates the optimal m for each channel based on the mean absolute residual (μ) of the data, using the approximation $m \approx \lceil 0.69 \cdot \mu \rceil$.

4.2 Usage

Encoding

The encoder takes an audio file (.wav) and produces a compressed file. The compression level can be controlled via optional parameters.

```
./audio_encoder -c <input.wav> <output.glmb> [options]

  -m <value>         Fixed Golomb parameter m (default: adaptive)
                     Use 0 for adaptive mode (recommended)

  -p <0|1|2|3>       Predictor (default: 1)
                     0 = no predictor
                     1 = order 1 (x[n-1])
                     2 = order 2 (2*x[n-1] - x[n-2])
                     3 = order 3

  -s <0|1>           Sign mode (default: 1)
                     0 = sign-magnitude
                     1 = interleaving

  -ms <0|1>          Mid-Side for stereo (default: 1)
                     0 = do not use
                     1 = use
```

1. The input `.wav` file is read using `libsndfile`, and its 16-bit samples are stored.
2. For stereo files, samples are de-interleaved into ‘left‘ and ‘right‘ buffers.
3. If Mid-Side is enabled, the ‘toMidSide(left, right)‘ function is called, converting the `vector<short>` data into `vector<int>` ‘mid‘ and ‘side‘ channels.
4. The ‘calculateResiduals‘ function applies the chosen predictor (e.g., Order 1) to the `int` data, generating a vector of `int` residuals.
5. If in adaptive mode, the optimal m is calculated via ‘estimateOptimalM‘ for each channel.
6. A custom file header (`GLMB`) is written to the output file. This header stores all necessary metadata, including sample rate, channels, predictor type, M/S flag, and the (fixed or adaptive) m parameter(s).
7. The `GolombCoder` is initialised with the correct m.
8. All residuals are encoded one by one into a `BitStreamWriter`.
9. Finally, the complete data from the `BitStreamWriter` is written to the output file.

Decoding

The decoding process reverses all transformations applied during encoding, reconstructing the original audio signal with bit-exact precision. The decoder operates in a strictly deterministic manner, ensuring lossless reconstruction.

```
./audio_encoder -d audio.glmb decoded.wav
```

The `AudioCodec::decompress` function reverses the process done by the encoder:

1. The `GLMB` header is read from the input file.
2. All metadata (sample rate, channels, m, predictor type, M/S flag) is extracted.
3. The `GolombCoder` is initialised with the m value(s) read from the header.
4. The compressed data blob is read into a `BitStreamReader`.
5. The `GolombCoder::decode` function is called in a loop (once per frame) to read the bitstream and reconstruct the original `vector<int>` of residuals.
6. The ‘reconstructFromResiduals‘ function applies the inverse predictor to the residuals, reconstructing the `vector<int>` samples (e.g., ‘Sample[n] = Residual[n] + Sample[n-1]‘).
7. If the M/S flag is set, ‘fromMidSide‘ is called to convert the `int` Mid/Side channels back into `short` Left/Right channels.
8. The final `short` samples are interleaved (if stereo) and written to the output `.wav` file using `libsndfile`.

4.3 Results

Lossless Verification

Following the corrections for integer overflow and rounding errors outlined in the Methodology, the codec was verified to be fully lossless. All test files were compressed and then decompressed, and a byte-for-byte comparison confirmed that the resulting `.wav` files were identical to the originals across all predictor, m, and Mid-Side configurations.

Compression Performance

The default configuration (Adaptive m, Predictor Order 1, Mid-Side enabled) provides robust compression across all test files. The performance on a selection of files is summarised in Table 4.1.

Audio File	Original Size	Compressed Size	Ratio	BPS (16-bit source)
`sample01.wav`	5,176,796	3,886,356	1.33:1	12.01
`sample02.wav`	4,519,484	3,599,258	1.26:1	12.75
`sample03.wav`	3,530,396	2,477,764	1.42:1	11.23
`sample06.wav`	4,235,996	2,500,202	1.69:1	9.44

Table 4.1: Codec performance with default settings on various audio files.

Analysis of Codec Parameters

To justify the chosen methodology, the key parameters of the codec were tested individually using `sample01.wav`. The results, shown in Table 4.2, demonstrate the impact of each stage.

Test	Configuration	Compressed Size	BPS
Default	**Adaptive m, Pred. 1, M/S**	**3,886,356**	**12.01**
No Prediction	Adaptive m, **Pred. 0**, M/S	4,778,599	14.77
No M/S	Adaptive m, Pred. 1, **No M/S**	3,956,019	12.23
Bad Fixed m	$m = 32$, Pred. 1, M/S	12,727,379	39.34

Table 4.2: Impact of codec parameters on `sample01.wav`.

The analysis leads to clear conclusions:

- **Prediction** (Default vs. Predictor 0) is the most critical stage, responsible for a 2.76 BPS reduction (saving ≈900KB) by removing temporal redundancy.
- **Mid-Side** (Default vs. No M/S) provides a consistent, secondary gain (0.22 BPS, saving ≈70KB) by removing inter-channel redundancy.
- **Adaptive** m (Default vs. $m = 32$) is essential. A poorly chosen fixed m results in catastrophic file expansion (from 12.01 BPS to 39.34 BPS). The adaptive algorithm correctly identifies the optimal parameter and is fundamental to the codec's success.

Chapter 5

Part 4

In Part IV, we implemented a lossless image codec for grayscale images, based on Golomb coding of the prediction residuals. This codec provides the best possible compression, using appropriate predictors and values of **m** for the Golomb codes.

5.1 Methodology

The lossless codec for grayscale images was implemented based on predictive coding combined with Golomb entropy encoding. This approach aims to reduce spatial redundancy in the image before encoding.

The core algorithm operates on a pixel-by-pixel basis in raster-scan order (left-to-right, top-to-bottom). For each pixel, a prediction P is calculated based on its already-processed neighbors. The predictor used is the JPEG-LS (LOCO-I) predictor #7, which is defined as $P = A + B - C$, where A is the pixel to the immediate left, B is the pixel directly above, and C is the pixel to the top-left.

To handle the image boundaries where neighbors are not available, the following edge cases were defined:

- **First Pixel (0, 0):** The prediction P is set to 0.
- **First Row (row = 0, col > 0):** The prediction P is set to A (the pixel to the left).
- **First Column (col > 0, row = 0):** The prediction P is set to B (the pixel above).

The calculated prediction P is clamped to the valid 8-bit pixel range $[0, 255]$.

Next, the prediction residual (or error) e is calculated by subtracting the prediction from the actual pixel value: $e = \text{PixelValue} - P$. This residual e now represents the "new information" or error in the prediction, and its value can range from -255 to $+255$.

To ensure the image can be decoded, the output file format includes a simple header. The image's dimensions (height, then width) are written as two 16-bit unsigned integers to the beginning of the bitstream, followed by the continuous stream of Golomb-encoded residuals.

5.2 Usage

Encoding

The encoder takes an image file (.ppm) and produces a compressed binary file. The compression level can be controlled via optional parameters.

```
./image_codec <inputImage.ppm> <outputFile.bin> <m>

# Example: Compress 'monarch.ppm' with a m value of 64
./image_codec -e /imagens_PPM/monarch.ppm monarch_enc.bin
```

Figure 5.1: Monarch image original

Decoding

The decoder takes a compressed binary file and grayscales the original PPM file.

NOTE: Because the PPM file format is strictly for color images (a 3 channel format), in this case the output file is going to be in PNG format, which is flexible enough to grayscale (1 channel).

```
./image_codec -d <inputFile.bin> <outputImage.png>

# Example: Decompress 'monarch_enc.bin' into 'monarch_dec.png'
./image_codec -d monarch_enc.bin monarch_dec.png
```

Figure 5.2: Monarch image gray scaled

5.3 Results

The batch-testing tool was executed on a directory of five 8-bit grayscale '.ppm' images. The results provide a clear analysis of the codec's performance and its sensitivity to the Golomb parameter m.

A summary of the best-achieved compression for each image is presented below:

Table 5.1: Best compression results per image from the test set.

Image Name	Original Size (Bytes)	Best m	Compressed Size (Bytes)	Best Compression Ratio
girl.ppm	262,144	16	171,204	1.5312
lena.ppm	262,144	16	171,968	1.5244
monarch.ppm	393,216	16	258,145	1.5232
airplane.ppm	262,144	16	173,107	1.5143
house.ppm	65,536	16	52,075	1.2586

Two primary conclusions can be drawn from the data:

1. **Optimal m Parameter:** For *every single image* tested, the optimal compression ratio was achieved with $\boldsymbol{m = 16}$. As the value of m increased, the compression ratio consistently decreased. For m values of 128 and 256, the ratio frequently fell below 1.0 (e.g., 0.8889 for 'lena.ppm' at $m = 256$), indicating that the "compressed" file was significantly larger than the original. This strongly suggests the distribution of the mapped residuals is sharply peaked at zero and is best modeled by a Golomb-Rice code with $k = 4$ (since $m = 2^k = 16$).

2. **Processing Time:** A clear, direct correlation exists between the m parameter and processing time. As m increases, both encoding and decoding times rise. For

example, encoding `monarch.ppm` took 58ms at $m = 16$ but increased to 92ms at $m = 256$. This is likely because larger m values result in less efficient (longer) average codewords, increasing the I/O load on the bitstream.

In summary, the implemented predictive codec is effective, achieving compression ratios between 1.25 and 1.53 on the test set. Its performance is critically dependent on the m parameter, with $m = 16$ proving to be the optimal choice among the tested values, delivering both the highest compression and the fastest execution time.